

Engineering Sprint 8 – Designing Asynchronous Workflows

Objective:

The objective of this sprint is to learn how Salesforce performs long-running tasks in the background using Asynchronous Apex. It focuses on improving application performance by moving non-critical operations outside the main transaction with Queueable Apex, Future Methods, Batch Apex, and Scheduled Apex.

Introduction:

Asynchronous processing enables Salesforce to execute time-consuming operations after the user's request has finished. This reduces waiting time, improves scalability, and ensures that large or complex tasks are completed efficiently without affecting the user experience.

Code:

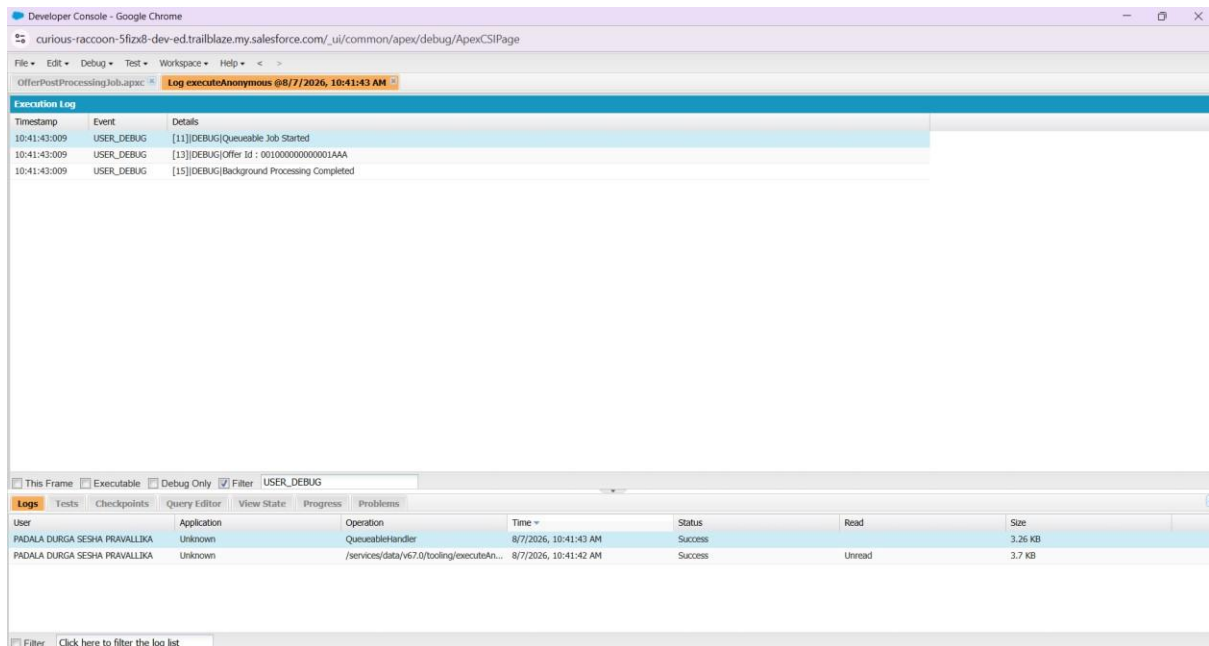
```
public class OfferPostProcessingJob implements Queueable {  
    private Id offerId;    public  
    OfferPostProcessingJob(Id offerId){  
        this.offerId = offerId;  
    }  
    public void execute(QueueableContext context){  
        System.debug('Job A');  
        System.enqueueJob(new NotificationJob());  
    }  
}
```

Execute Anonymous:

```
Id sampleId='0010000000000001AAA';  
System.enqueueJob(new OfferPostProcessingJob(sampleId));
```

Output

The Queueable job executes in the background and starts another Queueable job.



2. Future Method

Definition

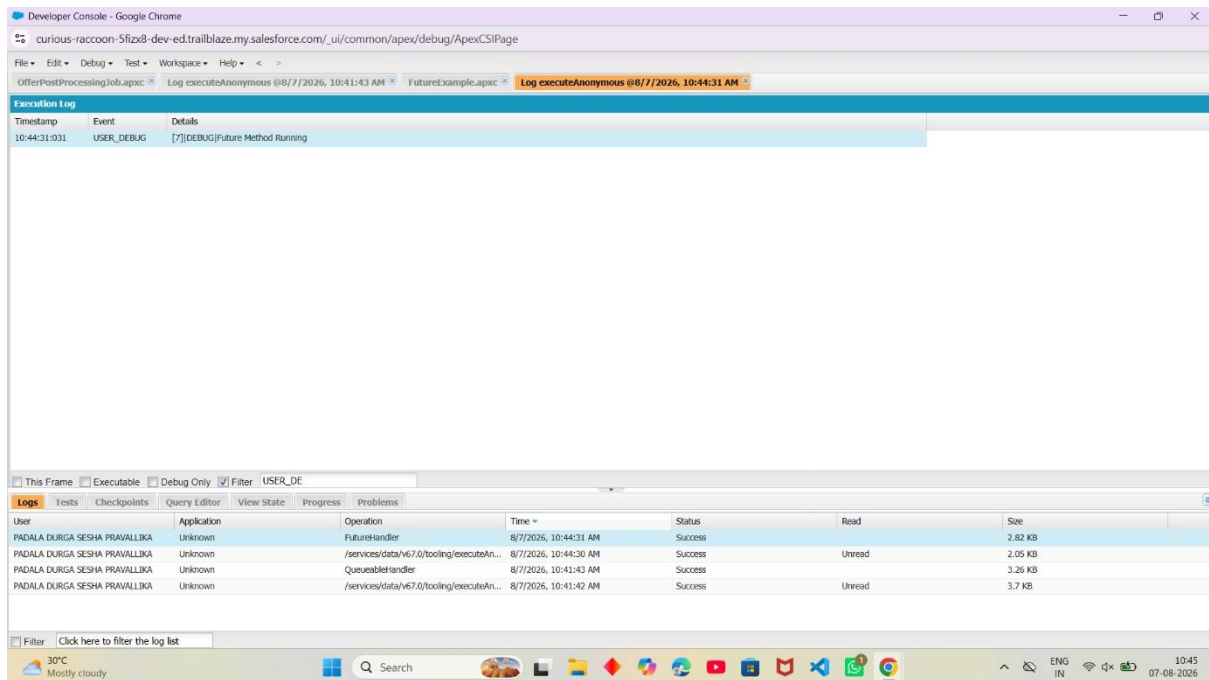
Future Methods are an asynchronous Apex feature used to execute background operations without delaying user actions. Existing Future Methods should be retained unless there is a justified technical or business requirement to replace them.

Code:

```
public class FutureExample {
    @future public static void
    sendNotification(){
        System.debug('Future Method Running');
    }
}
```

Execute Anonymous

```
FutureExample.sendNotification();
```



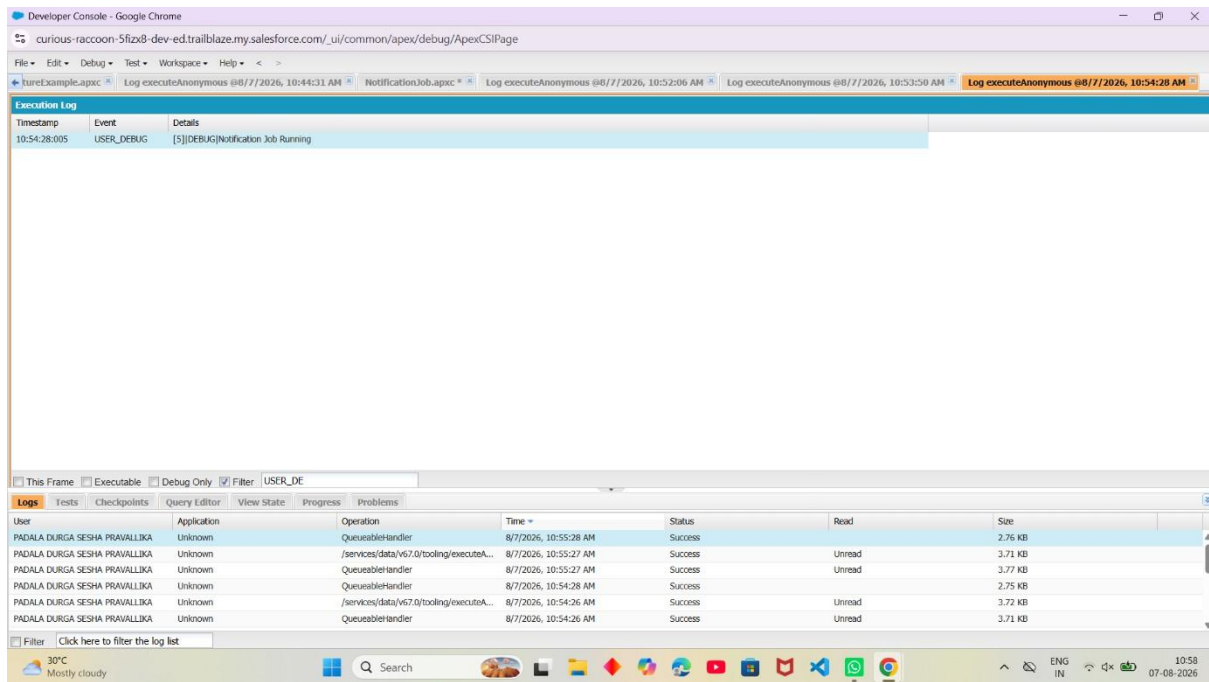
3. Queueable Chaining

Definition:

Queueable Chaining is a technique in Salesforce where one Queueable Apex job can enqueue another Queueable job after it completes. This helps execute tasks in a sequence, making complex asynchronous processes more organized, scalable, and easier to maintain.

Code:

```
public class NotificationJob implements Queueable{
    public void execute(QueueableContext context){
        System.debug('Notification Job Running');
    }
}
```



4. Batch Apex

Definition:

Batch Apex is an asynchronous processing feature in Salesforce that efficiently handles large amounts of data by dividing records into smaller batches. It manages the entire process using three key methods: start(), execute(), and finish().

Code:

```
public class StudentBatch implements Database.Batchable<SObject>{

    public Database.QueryLocator start(Database.BatchableContext bc){

        return Database.getQueryLocator(

            'SELECT Id,Name FROM Account'

        );

    }

    public void execute(Database.BatchableContext bc,List<Account> scope){

        for(Account a:scope){

            System.debug(a.Name);

        }

    }

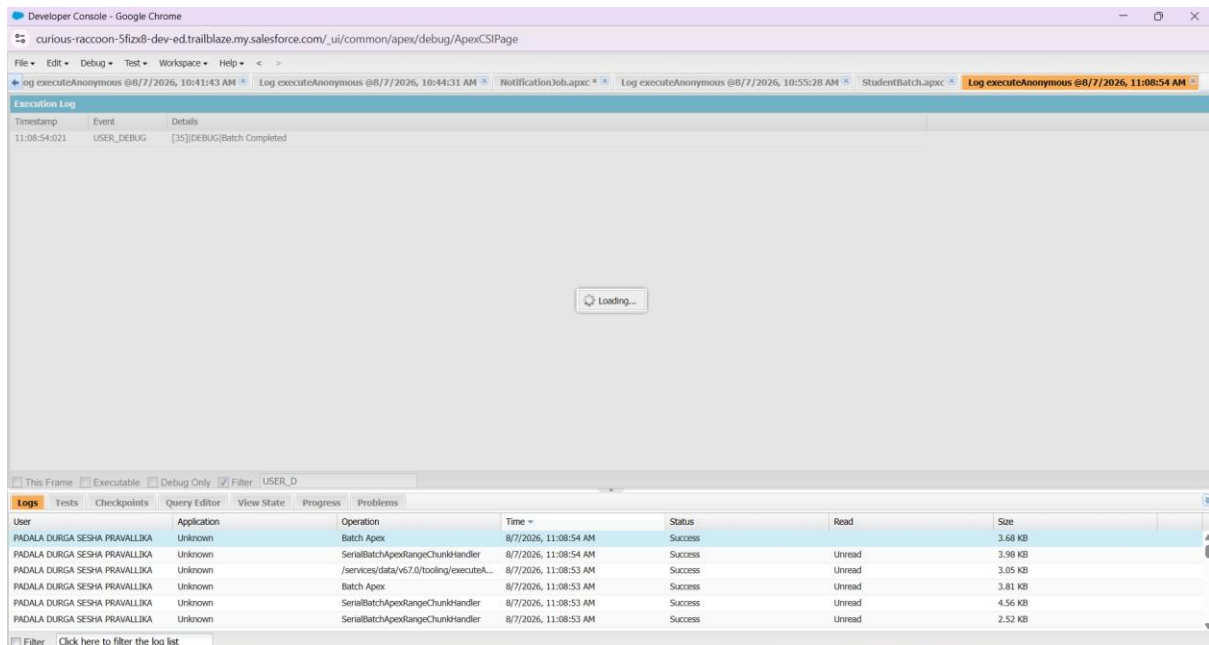
    public void finish(Database.BatchableContext bc){

        System.debug('Batch Completed');
```

```
}
}
```

Execute Anonymous

Database.executeBatch(new StudentBatch(),5);



5. Scheduled Apex

Definition:

Scheduled Apex is a Salesforce feature that allows Apex classes to run automatically at predefined dates and times using a **CRON expression**. It is commonly used to automate recurring tasks such as sending reports, updating records, and performing routine maintenance..

Code:

```
public class DailyScheduler implements Schedulable{
    public void execute(SchedulableContext sc){
        Database.executeBatch(new StudentBatch(),5);
    }
}
```

Execute Anonymous

String cron='0 0 12 * * ?';

System.schedule(

'Batch Scheduler',

cron, new

DailyScheduler());

The screenshot shows the Salesforce Setup page for 'Scheduled Jobs'. The left sidebar contains a navigation menu with options like Feature Settings, Sales, Products, Scheduled Reminders, Service, Field Service, Environments, and Jobs. The main content area is titled 'All Scheduled Jobs' and includes a summary card showing 'Percentage of Scheduled Jobs Used: 1%'. Below this is a table of scheduled jobs with columns for Action, Job Name, Submitted By, Submitted, Started, Next Scheduled Run, Type, and Cron Trigger ID. The table lists two jobs: 'Daily Job' and 'Metalytics Data Loader Job for Org: 00Dhg000002ho7'. The bottom of the page shows a Windows taskbar with various application icons and system information.

6. Apex Jobs Monitoring

Definition

The **Apex Jobs** page in Salesforce is used to monitor and manage asynchronous Apex processes. It allows developers to track the status of jobs, such as **Queued**, **Processing**, **Completed**, or **Failed**, making it easier to identify errors and troubleshoot issues.

The screenshot shows the Salesforce Setup page for 'Apex Jobs'. The left sidebar contains a navigation menu with options like Email, Custom Code, Apex Classes, Apex Settings, Apex Triggers, Application Test Execution, Application Test History, Environments, and Jobs. The main content area is titled 'Apex Jobs' and includes a summary card showing 'View: All'. Below this is a table of Apex jobs with columns for Action, Submitted Date, Job Type, Status, Status Detail, Total Batches, Batches Processed, Failures, Submitted By, Completion Date, Apex Class, Apex Method, and Apex Job ID. The table lists 14 jobs, including 'Scheduled Apex', 'Batch Apex', and 'StudentBatch'. The bottom of the page shows a Windows taskbar with various application icons and system information.